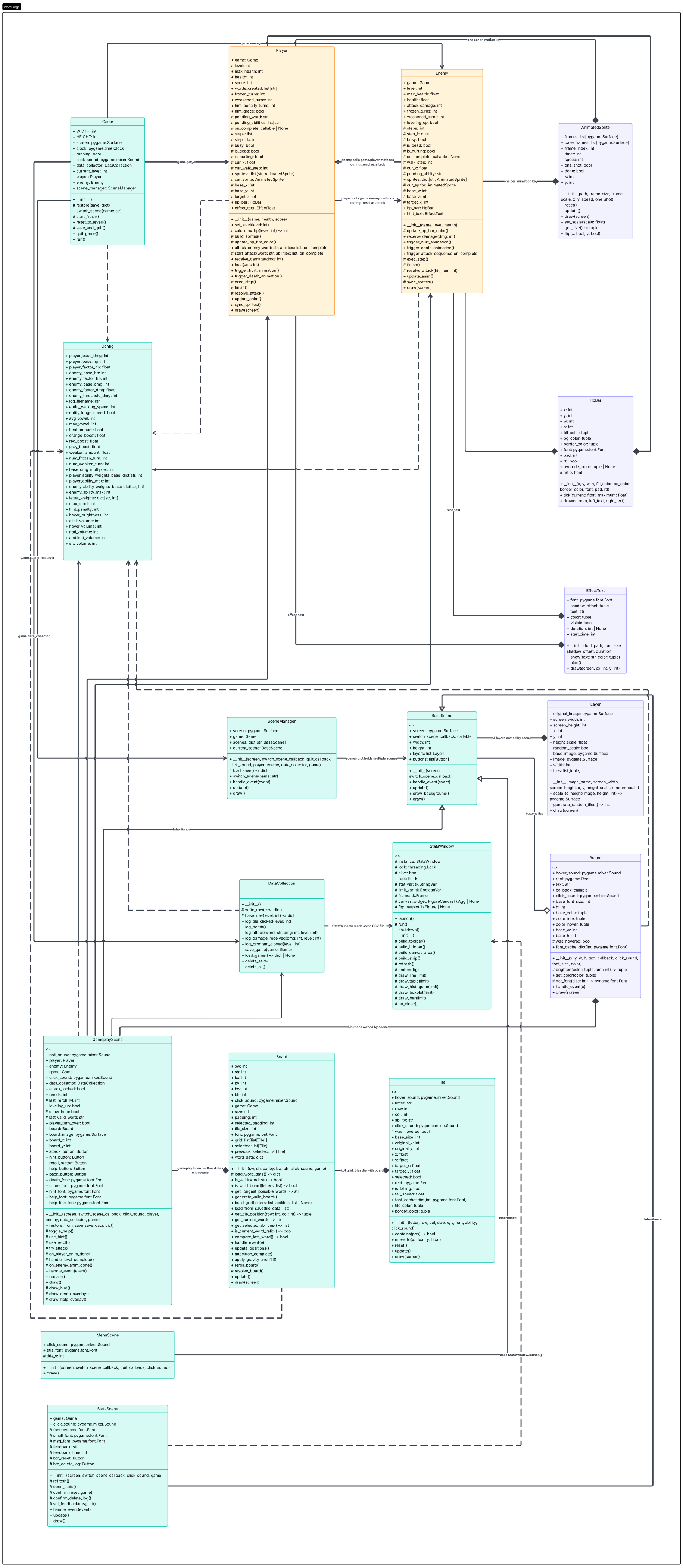
WordForge
Player
+ game: Game
# level: int
+ max_health: int
+ health: int
+ score: int
+ words_created: list[str]
+ frozen_turns: int
+ weakened_turns: int
+ hint_penalty_turns: int
+ hint_grace: bool
# pending_word: str
# pending_abilities: list[str]
# on_complete: callable | None
# steps: list
# step_idx: int
# busy: bool
# is_dead: bool
# is_hurting: bool
# cur_x: float
# cur_walk_step: int
+ sprites: dict[str, AnimatedSprite]
# cur_sprite: AnimatedSprite
# base_x: int
# base_y: int
# target_x: int
+ hp_bar: HpBar
+ effect_text: EffectText
+ __init__(game, health, score)
+ set_level(level: int)
# calc_max_hp(level: int) -> int
# build_sprites()
# update_hp_bar_color()
+ attack_enemy(word: str, abilities: list, on_complete)
# start_attack(word: str, abilities: list, on_complete)
+ receive_damage(dmg: int)
+ heal(amt: int)
+ trigger_hurt_animation()
+ trigger_death_animation()
# exec_step()
# finish()
# resolve_attack()
+ update_anim()
# sync_sprites()
+ draw(screen)
Enemy
+ game: Game
+ level: int
+ max_health: float
+ health: float
+ attack_damage: int
+ frozen_turns: int
+ weakened_turns: int
+ leveling_up: bool
# steps: list
# step_idx: int
# busy: bool
# is_dead: bool
# is_hurting: bool
# on_complete: callable | None
# walk_step: int
# cur_x: float
# pending_ability: str
+ sprites: dict[str, AnimatedSprite]
# cur_sprite: AnimatedSprite
# base_x: int
# base_y: int
# target_x: int
+ hp_bar: HpBar
+ hint_text: EffectText
+ __init__(game, level, health)
# update_hp_bar_color()
+ receive_damage(dmg: int)
+ trigger_hurt_animation()
+ trigger_death_animation()
+ trigger_attack_sequence(on_complete)
# exec_step()
# finish()
# resolve_attack(hit_num: int)
+ update_anim()
# sync_sprites()
+ draw(screen)
AnimatedSprite
+ frames: list[pygame.Surface]
+ base_frames: list[pygame.Surface]
+ frame_index: int
+ timer: int
+ speed: int
+ one_shot: bool
+ done: bool
+ x: int
+ y: int
+ __init__(path, frame_size, frames, scale, x, y, speed, one_shot)
+ reset()
+ update()
+ draw(screen)
+ set_scale(scale: float)
+ get_size() -> tuple
+ flip(x: bool, y: bool)
Game
+ WIDTH: int
+ HEIGHT: int
+ screen: pygame.Surface
+ clock: pygame.time.Clock
+ running: bool
+ click_sound: pygame.mixer.Sound
+ data_collector: DataCollection
+ current_level: int
+ player: Player
+ enemy: Enemy
+ scene_manager: SceneManager
+ __init__()
# restore(save: dict)
+ switch_scene(name: str)
# start_fresh()
+ reset_to_level1()
# save_and_quit()
+ quit_game()
+ run()
game.enemy
one per animation key
game.player
enemy calls game.player methods during _resolve_attack
player calls game.enemy methods during _resolve_attack
one per animation key
Config
+ player_base_dmg: int
+ player_base_hp: int
+ player_factor_hp: float
+ enemy_base_hp: int
+ enemy_factor_hp: int
+ enemy_base_dmg: int
+ enemy_factor_dmg: float
+ enemy_threshold_dmg: int
+ log_filename: str
+ entity_walking_speed: int
+ entity_lunge_speed: float
+ avg_vowel: int
+ max_vowel: int
+ heal_amount: float
+ orange_boost: float
+ red_boost: float
+ gray_boost: float
+ weaken_amount: float
+ num_frozen_turn: int
+ num_weaken_turn: int
+ base_dmg_multiplier: int
+ player_ability_weights_base: dict[str, int]
+ player_ability_max: int
+ enemy_ability_weights_base: dict[str, int]
+ enemy_ability_max: int
+ letter_weights: dict[str, int]
+ max_reroll: int
+ hint_penalty: int
+ hover_brightness: int
+ click_volume: int
+ hover_volume: int
+ noti_volume: int
+ ambient_volume: int
+ sfx_volume: int
HpBar
+ x: int
+ y: int
+ w: int
+ h: int
+ fill_color: tuple
+ bg_color: tuple
+ border_color: tuple
+ font: pygame.font.Font
+ pad: int
+ rtl: bool
+ override_color: tuple | None
# ratio: float
+ __init__(x, y, w, h, fill_color, bg_color, border_color, font, pad, rtl)
+ tick(current: float, maximum: float)
+ draw(screen, left_text, right_text)
hint_text
game.scene_manager
game.data_collector
effect_text
EffectText
+ font: pygame.font.Font
+ shadow_offset: tuple
+ text: str
+ color: tuple
+ visible: bool
+ duration: int | None
+ start_time: int
+ __init__(font_path, font_size, shadow_offset, duration)
+ show(text: str, color: tuple)
+ hide()
+ draw(screen, cx: int, y: int)
SceneManager
+ screen: pygame.Surface
+ game: Game
+ scenes: dict[str, BaseScene]
+ current_scene: BaseScene
+ __init__(screen, switch_scene_callback, quit_callback, click_sound, player, enemy, data_collector, game)
# load_save() -> dict
+ switch_scene(name: str)
+ handle_event(event)
+ update()
+ draw()
scenes dict holds multiple scenes
BaseScene
<<abstract>>
+ screen: pygame.Surface
+ switch_scene_callback: callable
+ width: int
+ height: int
+ layers: list[Layer]
+ buttons: list[Button]
+ __init__(screen, switch_scene_callback)
+ handle_event(event)
+ update()
+ draw_background()
+ draw()
4 layers owned by scene
Layer
+ original_image: pygame.Surface
+ screen_width: int
+ screen_height: int
+ x: int
+ y: int
+ height_scale: float
+ random_scale: bool
+ base_image: pygame.Surface
+ image: pygame.Surface
+ width: int
+ tiles: list[tuple]
+ __init__(image_name, screen_width, screen_height, x, y, height_scale, random_scale)
+ scale_to_height(image, height: int) -> pygame.Surface
+ generate_random_tiles() -> list
+ draw(screen)
buttons list
Inheritance
StatsWindow
<<singleton>>
instance: StatsWindow
lock: threading.Lock
alive: bool
+ root: tk.Tk
# stat_var: tk.StringVar
# limit_var: tk.BooleanVar
# frame: tk.Frame
# canvas_widget: FigureCanvasTkAgg | None
# fig: matplotlib.Figure | None
+ launch()
# run()
+ shutdown()
+ __init__()
# build_toolbar()
# build_infobar()
# build_canvas_area()
# build_strip()
# refresh()
# embed(fig)
# draw_line(limit)
# draw_table(limit)
# draw_histogram(limit)
# draw_boxplot(limit)
# draw_bar(limit)
# on_close()
DataCollection
+ __init__()
# write_row(row: dict)
# base_row(level: int) -> dict
+ log_tile_clicked(level: int)
+ log_death()
+ log_attack(word: str, dmg: int, level: int)
+ log_damage_received(dmg: int, level: int)
+ log_program_closed(level: int)
+ save_game(game: Game)
+ load_game() -> dict | None
+ delete_save()
+ delete_all()
StatsWindow reads same CSV file
Button
<<ui>>
+ hover_sound: pygame.mixer.Sound
+ rect: pygame.Rect
+ text: str
+ callback: callable
+ click_sound: pygame.mixer.Sound
+ base_font_size: int
+ h: int
+ base_color: tuple
+ color_idle: tuple
+ color_hover: tuple
+ base_w: int
+ base_h: int
# was_hovered: bool
+ font_cache: dict[int, pygame.font.Font]
+ __init__(x, y, w, h, text, callback, click_sound, font_size, color)
# brighten(color: tuple, amt: int) -> tuple
+ set_color(color: tuple)
# get_font(size: int) -> pygame.font.Font
+ handle_event(e)
+ draw(screen)
6 buttons owned by scene
GameplayScene
<<scene>>
+ noti_sound: pygame.mixer.Sound
+ player: Player
+ enemy: Enemy
+ game: Game
+ click_sound: pygame.mixer.Sound
+ data_collector: DataCollection
+ attack_locked: bool
+ rerolls: int
# last_reroll_lvl: int
+ leveling_up: bool
# show_help: bool
# last_valid_word: str
+ player_turn_over: bool
+ board: Board
+ board_image: pygame.Surface
+ board_x: int
+ board_y: int
+ attack_button: Button
+ hint_button: Button
+ reroll_button: Button
+ help_button: Button
+ back_button: Button
+ death_font: pygame.font.Font
+ score_font: pygame.font.Font
+ hint_font: pygame.font.Font
+ help_font: pygame.font.Font
+ help_title_font: pygame.font.Font
+ __init__(screen, switch_scene_callback, click_sound, player, enemy, data_collector, game)
+ restore_from_save(save_data: dict)
# toggle_help()
# use_hint()
# use_reroll()
# try_attack()
# on_player_anim_done()
# handle_level_complete()
# on_enemy_anim_done()
+ handle_event(event)
+ update()
+ draw()
# draw_hud()
# draw_death_overlay()
# draw_help_overlay()
gameplay.board — Board dies with scene
Board
+ sw: int
+ sh: int
+ bx: int
+ by: int
+ bw: int
+ bh: int
+ click_sound: pygame.mixer.Sound
+ game: Game
+ size: int
+ padding: int
+ selected_padding: int
+ tile_size: int
+ font: pygame.font.Font
+ grid: list[list[Tile]]
+ selected: list[Tile]
+ previous_selected: list[Tile]
+ word_data: dict
+ __init__(sw, sh, bx, by, bw, bh, click_sound, game)
# load_word_data() -> dict
+ is_valid(word: str) -> bool
+ is_valid_board(letters: list) -> bool
+ get_longest_possible_word() -> str
+ generate_valid_board()
+ build_grid(letters: list, abilities: list | None)
+ load_from_save(tile_data: list)
+ get_tile_position(row: int, col: int) -> tuple
+ get_current_word() -> str
+ get_selected_abilities() -> list
+ is_current_word_valid() -> bool
+ compare_last_word() -> bool
+ handle_event(e)
+ update_positions()
+ attack(on_complete)
+ apply_gravity_and_fill()
+ reroll_board()
# resolve_board()
+ update()
+ draw(screen)
5x5 grid, tiles die with board
Tile
<<ui>>
+ hover_sound: pygame.mixer.Sound
+ letter: str
+ row: int
+ col: int
+ ability: str
+ click_sound: pygame.mixer.Sound
# was_hovered: bool
+ base_size: int
+ original_x: int
+ original_y: int
+ x: float
+ y: float
+ target_x: float
+ target_y: float
+ selected: bool
+ rect: pygame.Rect
+ is_falling: bool
+ fall_speed: float
+ font_cache: dict[int, pygame.font.Font]
+ tile_color: tuple
+ border_color: tuple
+ __init__(letter, row, col, size, x, y, font, ability, click_sound)
+ contains(pos) -> bool
+ move_to(x: float, y: float)
+ reset()
+ update()
+ draw(screen)
Inheritance
Inheritance
MenuScene
+ click_sound: pygame.mixer.Sound
+ title_font: pygame.font.Font
# title_y: int
+ __init__(screen, switch_scene_callback, quit_callback, click_sound)
+ draw()
calls StatsWindow.launch()
StatsScene
+ game: Game
+ click_sound: pygame.mixer.Sound
# font: pygame.font.Font
# small_font: pygame.font.Font
# msg_font: pygame.font.Font
# feedback: str
# feedback_time: int
# btn_reset: Button
# btn_delete_log: Button
+ __init__(screen, switch_scene_callback, click_sound, game)
# refresh()
# open_stats()
# confirm_reset_game()
# confirm_delete_log()
# set_feedback(msg: str)
+ handle_event(event)
+ update()
+ draw()